Reproducing Vermaseren's numerical-instability claim for $\gamma^*\gamma^* \to \mu^+\mu^-$

nikhef-form-lab, Part 5 (`06-mc-multiperipheral`)

November 11, 2026

Abstract

J.A.M. Vermaseren's *Two Photon Processes at Very High Energies* (NIKHEF-H/82-15, 1982) states, discussing the reaction $e^+e^- \to e^+e^-\mu^+\mu^-$ at PETRA/LEP energies: "If the matrix element were to be completely evaluated in the standard fashion – i.e. expressing it completely in terms of 4-vector products and then substituting their numerical values for each Monte Carlo generated point in phase space – then the cancellations between the various terms would be so bad that even the 60 bit accuracy of a CDC computer would not suffice." We take that statement as given and ask how much of it can be reproduced for the isolated two-photon subprocess $\gamma^*\gamma^* \to \mu^+\mu^-$.

What we do reproduce: the squared matrix element by the naive ("standard fashion") route and by Vermaseren's Schouten-identity / Levi-Civita-stabilized route (3.5); symbolic identity of the two; IEEE double digit-loss of the naive 98-term poly against long double and against (3.5); and the electron-dressed (III.11) layer ($\sum M_{ij} \equiv L \otimes L$, including FORM transcription and a `gamgam/peripp` gate).

Limitation (not a revision of Vermaseren): for free photon polarizations on the subprocess poly alone, catastrophic double/long-double loss tracks the Ward surface $e_1 \to q_1$, not soft virtuality $t_1, t_2 \to 0$ by itself (physical-transverse MC stays healthy down to $\varepsilon = 10^{-12}$). Soft-$|t|$ collapse of a dressed L⊗L(naive) path versus $\sum M_{ij}$ appears on real $2 \to 4$ `gamgam` samples. We do not claim a full PETRA/LEP e^+e^- Monte Carlo with CDC-era arithmetic.

Contents

1 The process and the two diagrams **2**
1.1 Kinematic conventions . . . 3

2 Feynman rules and the amplitude **3**

3 The naive ("standard fashion") squared amplitude **4**
3.1 Split of N into four pieces . . . 4
3.2 Spin sum as a 4×4 of traces . . . 4
3.3 Worked example: T_{22} . . . 4
3.4 Result of the full sum . . . 5
3.5 Assumptions, and what falsifies them . . . 5
3.6 Checks that can fail . . . 6

4 Vermaseren's stable form **6**
4.1 Common-denominator, gauge-safe amplitude (III.3) . . . 6
4.2 Squared form and the M_{ij} decomposition (III.10) . . . 7

4.3 Denominator: analogous to `pi0.c`'s `la`, not identical . . . 7
4.4 What this section does and does not claim . . . 8

5 Symbolic cross-check (FORM) 8
5.1 What is checked . . . 8
5.2 Naive ≡ stable subreaction (3.5) . . . 8
5.3 M_{ij} of (III.11) vs $W_i \otimes W_j$ dressing . . . 9
5.4 What is not yet claimed . . . 9

6 Numerical demonstration of the cancellation 9
6.1 Stress configuration . . . 9
6.2 Scan . . . 9
6.3 What this section does and does not claim . . . 10

7 Monte Carlo scan 10
7.1 Setup . . . 10
7.2 Results . . . 11
7.3 What this section does and does not claim . . . 11

8 Conclusion 11

A Full naive derivation: subtleties and term dump 12
A.1 Why the main text stops at one T_{ij} . . . 12
A.2 Bugs and failed checks (kept visible) . . . 12
A.3 Per-piece structure (from the generator) . . . 12
A.4 Code layout . . . 13

B Verification of (3.5) and M_{ij} 14
B.1 Two different Q's in the paper . . . 14
B.2 Transcribed (3.5) used by the checker . . . 15
B.3 Scanned-sign correction on the last two bilinears . . . 15
B.4 $M_{ij}/B^2 \equiv W_i \otimes W_j$ method . . . 15
B.5 Inventory of falsifiable gates . . . 16

C Generated term dump 16

1 The process and the two diagrams

We study the subprocess $\gamma^*(q_1)\,\gamma^*(q_2) \to \mu^-(p_6)\,\mu^+(p_7)$, following Vermaseren's Fig. 3 labeling (reproduced in Figures 1–2). This is the matrix-element core of the full $e^+e^- \to e^+e^-\mu^+\mu^-$ two-photon process (Fig. 1 of the same paper): the electron and positron each radiate a virtual photon (q_1, q_2), which then fuse via muon exchange into the $\mu^+\mu^-$ pair. The electron/positron lines and the two photon propagators $1/q_1^2$, $1/q_2^2$ are common to both diagrams; the cancellations Vermaseren warns about live in the squared $\gamma^*\gamma^* \to \mu^+\mu^-$ matrix element, so we isolate that subprocess first and treat the dressed e^+e^- layer separately.

There are exactly two tree diagrams, differing by which photon attaches to the muon line first:

- **Diagram** k_1: photon q_1 attaches first; the internal muon propagator carries momentum $k_1 = p_6 - q_1 = q_2 - p_7$.

- **Diagram** k_2 (crossed): photon q_2 attaches first; the internal muon propagator carries momentum $k_2 = p_6 - q_2 = q_1 - p_7$.

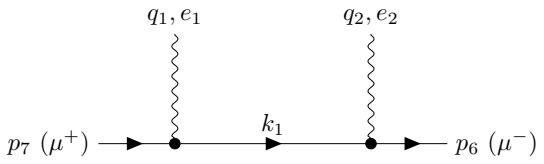


Figure 1: Diagram k_1: q_1 attaches to the muon line first.

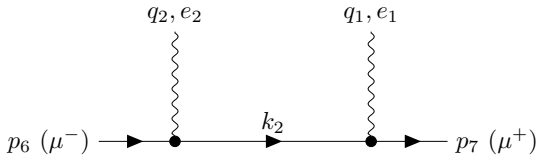


Figure 2: Diagram k_2 (crossed): q_2 attaches to the muon line first.

1.1 Kinematic conventions

On-shell conditions: $p_6^2 = p_7^2 = m^2$ (muon mass), while the photons are virtual with $q_1^2 = t_1$, $q_2^2 = t_2$ (generally $t_1, t_2 \neq 0$, since these are internal propagators of the full process, not real external photons). Momentum conservation at the $\gamma^*\gamma^*$ vertex reads $q_1 + q_2 = p_6 + p_7 \equiv Q$. We will also use $k_1^2 = (p_6 - q_1)^2$ and $k_2^2 = (p_6 - q_2)^2$, with propagator denominators $D_1 \equiv k_1^2 - m^2$, $D_2 \equiv k_2^2 - m^2$.

The "multiperipheral peak" regime the paper's warning is about is $t_1, t_2 \to 0$ (both photons nearly on-shell/real) at fixed, large $Q^2 = s$ – the same corner of phase space studied numerically for the π^0 case in Topic 5.

2 Feynman rules and the amplitude

Using standard QED Feynman rules (photon-fermion vertex $-ie\gamma^\mu$, fermion propagator $i(\not{k} + m)/(k^2 - m^2)$):

$$\mathcal{M}_1 = (-ie)^2\, \bar{u}(p_6)\, (-i\not{\epsilon}_1)\, \frac{i(\not{k}_1 + m)}{D_1}\, (-i\not{\epsilon}_2)\, v(p_7) \tag{1}$$

$$\mathcal{M}_2 = (-ie)^2\, \bar{u}(p_6)\, (-i\not{\epsilon}_2)\, \frac{i(\not{k}_2 + m)}{D_2}\, (-i\not{\epsilon}_1)\, v(p_7) \tag{2}$$

Dropping the overall coupling and bookkeeping factors of i into a common prefactor (matching Vermaseren's own convention, cf. formula (III.3) of `twophoton.pdf`), the total amplitude is

$$\mathcal{M} \;\propto\; \bar{u}(p_6)\, N\, v(p_7), \qquad N \equiv \not{\epsilon}_1\, \frac{\not{k}_1 + m}{D_1}\, \not{\epsilon}_2 \;+\; \not{\epsilon}_2\, \frac{\not{k}_2 + m}{D_2}\, \not{\epsilon}_1 \tag{3}$$

This matches the left-hand side of formula (III.3) in `twophoton.pdf` exactly, confirming the starting point before proceeding to the trace algebra.

3 The naive ("standard fashion") squared amplitude

This section carries out the calculation the paper warns about: expand N into pieces, form the spin-summed squared amplitude as a sum of Dirac traces, reduce every trace to products of 4-vector dots, and keep the propagator denominators D_1 and D_2 as they stand. No Schouten identity and no Levi-Civita rearrangement is used. The algebra is implemented in `scripts/mumu_trace_lib.py` and driven by `scripts/mumu_naive_trace.py`; the checks that can falsify it are listed in Section 3.6. Full term listings and the subtleties that showed up while deriving them are in Appendix A.

3.1 Split of N into four pieces

Write each diagram's numerator as a massless slash piece plus a mass piece:

$$N_1 = \frac{\not\epsilon_1 \not k_1 \not\epsilon_2}{D_1}, \qquad N_2 = \frac{m \not\epsilon_1 \not\epsilon_2}{D_1},$$
$$N_3 = \frac{\not\epsilon_2 \not k_2 \not\epsilon_1}{D_2}, \qquad N_4 = \frac{m \not\epsilon_2 \not\epsilon_1}{D_2}, \tag{4}$$

so $N = N_1 + N_2 + N_3 + N_4$. For real polarization vectors the conjugate amplitude $\bar{N}$ is obtained by reversing the order of the gamma matrices in each piece (e.g. $\overline{N_1} = \not\epsilon_2 \not k_1 \not\epsilon_1 / D_1$). This is an assumption of the calculation; it is the standard real-e convention, not something we re-derived from a complex polarization basis.

3.2 Spin sum as a 4×4 of traces

The spin-summed squared amplitude (up to the overall coupling stripped in Section 2) is

$$S = \mathrm{Tr}\Big[(\not p_6 + m)\, N\, (\not p_7 - m)\, \bar{N}\Big] = \sum_{i,j=1}^{4} T_{ij}, \qquad T_{ij} = \mathrm{Tr}\Big[(\not p_6 + m)\, N_i\, (\not p_7 - m)\, \overline{N_j}\Big]. \tag{5}$$

Each T_{ij} still contains the factor $1/(D_a D_b)$ from the two pieces involved. Expanding $(\not p_6 + m)$ and $(\not p_7 - m)$ gives four ordinary gamma-chain traces per T_{ij}. A chain with an odd number of gamma matrices has vanishing trace; that kills some of the four contributions outright (see the worked example below).

Reduction of an even-length chain uses the standard recursive contraction (with $\mathrm{Tr}[1] = 4$)

$$\mathrm{Tr}[\gamma^{v_1} \cdots \gamma^{v_n}] = \sum_{k=2}^{n} (-1)^{k-1}\, (v_1 \cdot v_k)\, \mathrm{Tr}[\text{chain without } v_1, v_k]. \tag{6}$$

There is no γ^5 in this QED amplitude, so no Levi-Civita tensor appears. That is by construction of the Feynman rules used in Section 2, not an extra truncation.

3.3 Worked example: T_{22}

Take $i = j = 2$, the shortest nontrivial diagonal piece:

$$T_{22} = \frac{m^2}{D_1^2}\, \mathrm{Tr}\Big[(\not p_6 + m)\, \not\epsilon_1 \not\epsilon_2\, (\not p_7 - m)\, \not\epsilon_2 \not\epsilon_1\Big]. \tag{7}$$

The four sub-traces are:

$$\begin{aligned} A &= \mathrm{Tr}[\not{p}_6 \not{\epsilon}_1 \not{\epsilon}_2 \not{p}_7 \not{\epsilon}_2 \not{\epsilon}_1], \\ B &= (-m)\,\mathrm{Tr}[\not{p}_6 \not{\epsilon}_1 \not{\epsilon}_2 \not{\epsilon}_2 \not{\epsilon}_1], \\ C &= m\,\mathrm{Tr}[\not{\epsilon}_1 \not{\epsilon}_2 \not{p}_7 \not{\epsilon}_2 \not{\epsilon}_1], \\ D &= (-m^2)\,\mathrm{Tr}[\not{\epsilon}_1 \not{\epsilon}_2 \not{\epsilon}_2 \not{\epsilon}_1]. \end{aligned} \tag{8}$$

B and C each have five gamma matrices, so $B = C = 0$. The four-gamma identity $\mathrm{Tr}[\not{a}\not{b}\not{c}\not{d}] = 4\big[(a{\cdot}b)(c{\cdot}d) - (a{\cdot}c)(b{\cdot}d) + (a{\cdot}d)(b{\cdot}c)\big]$ gives

$$D = (-m^2) \cdot 4\,(e_1{\cdot}e_1)(e_2{\cdot}e_2). \tag{9}$$

The six-gamma trace A is reduced by (6); after imposing the on-shell conditions $p_6{\cdot}p_6 = p_7{\cdot}p_7 = m^2$ (Section 3.5) one obtains

$$\begin{aligned} A = {} & 4(e_1{\cdot}e_1)(e_2{\cdot}e_2)(p_6{\cdot}p_7) - 8(e_1{\cdot}e_1)(e_2{\cdot}p_6)(e_2{\cdot}p_7) \\ & + 16(e_1{\cdot}e_2)(e_1{\cdot}p_6)(e_2{\cdot}p_7) - 8(e_1{\cdot}p_6)(e_1{\cdot}p_7)(e_2{\cdot}e_2). \end{aligned} \tag{10}$$

Putting A and D together with the prefactor m^2/D_1^2,

$$\begin{aligned} T_{22} = \frac{1}{D_1^2}\Big[& -4(e_1{\cdot}e_1)(e_2{\cdot}e_2)\,m^4 + 4(e_1{\cdot}e_1)(e_2{\cdot}e_2)\,m^2(p_6{\cdot}p_7) \\ & - 8(e_1{\cdot}e_1)(e_2{\cdot}p_6)(e_2{\cdot}p_7)\,m^2 + 16(e_1{\cdot}e_2)(e_1{\cdot}p_6)(e_2{\cdot}p_7)\,m^2 \\ & - 8(e_1{\cdot}p_6)(e_1{\cdot}p_7)(e_2{\cdot}e_2)\,m^2\Big]. \end{aligned} \tag{11}$$

This matches the symbolic output of `mumu_naive_trace.py` for `T[N2,N2]` exactly (term by term). The same script evaluates the other fifteen T_{ij}; those expressions are not re-derived by hand in the main text.

3.4 Result of the full sum

Running `scripts/mumu_naive_trace.py` (which uses the shared library above) yields:

- none of the sixteen T_{ij} vanishes as a whole;
- after expanding and collecting, the sum S contains **98** distinct monomials in external dot products;
- those monomials split by denominator as 18 with $1/D_1^2$, 18 with $1/D_2^2$, and 62 with $1/(D_1 D_2)$.

The separate D_1, D_2, and $D_1 D_2$ families are the concrete form of the paper's "standard fashion" expression: there is no single common denominator at this stage, and large cancellations between families (and within a family) are left for floating-point evaluation to discover. The machine-readable listing is `scripts/generated/mumu_naive_Tij.txt`, reproduced in Appendix C.

3.5 Assumptions, and what falsifies them

The following are imposed or not imposed, deliberately:

1. **Imposed:** $p_6{\cdot}p_6 = p_7{\cdot}p_7 = m^2$. These are the physical muon mass shells. If they are left as free symbols, the squared Ward identity of Section 3.6 fails (a nonzero remainder of order 10^2 monomials). So the on-shell conditions are not optional bookkeeping.

2. **Not imposed in S:** momentum conservation relating $\{e_1, e_2, k_1, k_2, p_6, p_7\}$, and the definitions $D_i = k_i^2 - m^2$. In the naive sum, all those dots and both D_i are independent symbols. That is the literal "express in 4-vector products" expansion; further kinematic reduction is a separate step and is *not* claimed here.

3. **Assumed:** real e_1, e_2 so that $\bar{N}$ is the gamma-order reverse of N. Not cross-checked against a complex-polarization formulation in this repository.

3.6 Checks that can fail

Two independent Ward-identity scripts test gauge invariance of the same N that enters S:

1. `scripts/mumu_gauge_check.py` substitutes $e_1 \to q_1$ in N alone, reduces with the on-shell Dirac equations $\bar{u}(p_6)\not{p}_6 = m\,\bar{u}(p_6)$ and $\not{p}_7\, v(p_7) = -m\, v(p_7)$, and substitutes $D_i = k_i^2 - m^2$. Result: diagram 1 alone $\to -\not{\epsilon}_2$, diagram 2 alone $\to +\not{\epsilon}_2$, sum $= 0$. Neither diagram is gauge invariant by itself; only the sum is. (An earlier draft of this script had a duplicated k_1 token in one word list; the hand reduction disagreed with the machine until that bug was fixed.)

2. `scripts/mumu_gauge_check_squared.py` substitutes $e_1 \to q_1$ *only* on the N side of (5), leaves $\bar{N}$ physical, and uses the same trace code as the naive sum. Result: the full expression vanishes. Replacing e_1 on both N and $\bar{N}$ would give the trivial $0 \cdot 0 = 0$ and is not used.

Both scripts exit with a clear failure message if the identity does not hold. As of this writing both pass. Index and run order: `scripts/README-mumu.md`.

4 Vermaseren's stable form

Section 3 expanded N into 4-vector products and left D_1 and D_2 as separate families. The paper's remedy (`twophoton.pdf`, Section III and Appendix C) is to rewrite the *sum* of the two diagrams so that gauge cancellations sit inside Levi-Civita tensors and a single common propagator denominator, then to square that form and reduce products of ε-tensors with the Schouten identity. The load-bearing structural claims are recorded here; the full Appendix C reduction chain for the cross pieces M_{12}, M_{21} is not re-derived term-by-term in this section (the paper itself calls that algebra "far from trivial" and checked it with Schoonschip). A machine identity between Section 3 and this section is left to Section 5.

4.1 Common-denominator, gauge-safe amplitude (III.3)

The starting point is the same N as in (3). Using the on-shell Dirac equations $\bar{u}(p_6)(\not{p}_6 - m) = 0$ and $(\not{p}_7 + m)v(p_7) = 0$, gamma-matrix algebra, and the Schouten identity in four dimensions, the paper rewrites the *sum* of the two diagrams as a single structure with denominator $D_1 D_2$ (formula (III.3)). Schematically,

$$\bar{u}(p_6)\, N\, v(p_7) = -\varepsilon^{q_1 e_1 \alpha \mu}\, \varepsilon_{q_1 e_1 \beta \mu}\, \frac{\bar{u}(p_6)\left(\gamma^\beta \not{p}_6 \gamma_\alpha + \chi\, \gamma_\alpha \not{p}_7 \gamma^\beta\right) v(p_7)}{D_1 D_2}, \tag{12}$$

where χ encodes the relative weight of the two spinor orderings in the paper's notation, and the precise index routing matches `twophoton.pdf`. The point that matters for numerics is structural:

- neither diagram alone is written in a form that vanishes under $e_1 \to q_1$; the Levi-Civita factors make the *sum* gauge-safe by construction (consistent with the Ward checks of Section 3.6);
- D_1 and D_2 appear only as the product $D_1 D_2$, not as separate $1/D_1^2$, $1/D_2^2$, and $1/(D_1 D_2)$ families.

Under (III.5) the paper states explicitly that remaining cancellations inside the Levi-Civita tensors are due to spin structure and are "not serious" compared with the naive 4-vector-product cancellations.

4.2 Squared form and the M_{ij} decomposition (III.10)

After squaring (III.3)/(III.4), tracing over muon spins, and attaching the electron/positron (or proton) currents in the two-photon factorization, the squared matrix element for two spin-$\frac{1}{2}$ point fermions is cast as (formula (III.10))

$$|\mathcal{M}|^2 = \frac{W_1^{(1)} W_1^{(2)} M_{11} + W_1^{(1)} W_2^{(2)} M_{12} + W_2^{(1)} W_1^{(2)} M_{21} + W_2^{(1)} W_2^{(2)} M_{22}}{\Big(q_1^2\, q_2^2 \left[(q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2 \right] \Big)^2}, \tag{13}$$

with $Q \equiv q_1 + q_2$. The weights W_1, W_2 are photon-vertex structure functions; for point particles the paper's normalization gives $W_1 = W_2 = 1$ (formula (III.9)), so (13) collapses to $(M_{11} + M_{12} + M_{21} + M_{22})$ over the displayed denominator.

The four pieces M_{ij} (formula (III.11)) are polynomials in dot products and Levi-Civita contractions of $\{q_1, q_2, Q, p_1\}$:

- M_{11} — no residual ε tensor ("pure QED" block);
- M_{12}, M_{21} — one ε structure each (need the Appendix C identities (C13)/(C14));
- M_{22} — the densest Levi-Civita content.

This is the same *recipe* as `pi0.c`'s split into `part1`.. `part4` (Gram determinant plus minors), but here the four blocks are cross terms in the (W_1, W_2) bilinears rather than minors of one Gram matrix.

4.3 Denominator: analogous to `pi0.c`'s `la`, not identical

The factor $(q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2$ in (13) looks like the quadratic bracket inside `pi0.c`'s `la`. As a pure vector identity (no on-shell relation),

$$(q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2 = -\,t_1\,(t_1 + 2\,q_1{\cdot}q_2), \qquad t_1 = q_1^2, \quad Q = q_1 + q_2, \tag{14}$$

which privileges q_1 through $Q{\cdot}q_1$. By contrast, `pi0.c`'s bracket $t_1 t_2 - (q_1{\cdot}q_2)^2$ is symmetric in t_1, t_2. On-shell, $q_1{\cdot}q_2 = (\mu^2 - t_1 - t_2)/2$, the two brackets become different polynomials in t_1, t_2, μ^2. This was checked in `scripts/muon_pair_denominator_check.py` and is recorded in Topic 6; an earlier draft that claimed a literal match was wrong. The two denominators are only *analogous* Gram-type invariants of double-t-channel kinematics.

4.4 What this section does and does not claim

1. **Claimed here:** the stable route replaces separate D_1, D_2 families by a common $D_1 D_2$ and hides gauge cancellation in ε tensors; the squared point-particle answer has the M_{ij} shape (13) with $W_i = 1$; the denominator bracket satisfies (14) and is not `pi0.c`'s `la`.

2. **Not claimed here:** an independent term-by-term re-derivation of Appendix C through (C13)/(C14), or a proof that the expanded M_{ij} equal the 98-term naive sum of Section 3. That identity is the job of Section 5.

5 Symbolic cross-check (FORM)

Section 3 used a Python Clifford contraction; this section reimplements the load-bearing identities in **FORM**, which is the right tool for Dirac traces at this scale (C is reserved for numeric digit-loss work in Section 6; a Rust CAS would not add domain leverage here).

5.1 What is checked

1. **Squared Ward identity in FORM** (`form/mumu_ward.frm`). Same non-trivial test as Section 3.6: substitute $e_1 \to q_1$ only in N (diagram 1: $q_1 = p_6 - k_1$; diagram 2: $q_1 = p_7 + k_2$), leave $\bar{N}$ physical, take $\mathrm{Tr}[(\not{p}_6 + m) N_g (\not{p}_7 - m) \bar{N}]$, clear $D_1^2 D_2^2$, set $D_i = k_i^2 - m^2$, impose $p_7 = p_6 - k_1 - k_2$ and $p_6^2 = m^2$. FORM prints `Sg = 0`. This is the gauge premise of the stable route (Section 4), now machine-checked in FORM rather than only in Python.

2. **Naive squared amplitude in FORM** (`form/mumu_naive.frm`). Same $N/\bar{N}$ split as Section 3, evaluated with `Trace4`. Provides an independent FORM baseline for later comparison to a transcribed (III.11).

Run from `form/`:

```
make check    # must print PASS: ... Sg = 0
make naive    # optional inspection of S
```

5.2 Naive ≡ stable subreaction (3.5)

The stable Levi-Civita form of the $\gamma^*\gamma^* \to \mu^+\mu^-$ *subreaction* (before attaching electron currents) is formula (3.5) of Nucl. Phys. B229 (1983) 347 — preprint (III.5). In that formula $Q = p_6 - p_7$ (not $q_1 + q_2$; the latter Q appears in (III.10)/(3.10)). The bracket $B = (q_1 \cdot q_2)^2 - (Q \cdot q_1)^2$ equals $D_1 D_2$ with the paper's D_i identities under (3.6).

Script `scripts/mumu_stable_35_check.py` evaluates (3.5) and compares it to the Section 3 poly; Appendix B records the transcribed formula, the $Q = p_6 - p_7$ convention, the Levi-Civita identities, the scanned-sign correction, and the PASS gate.

```
python3 scripts/mumu_stable_35_check.py   # must print PASS
```

This is the machine identity between Section 3 and the paper's stable *subreaction* form. It is not yet a transcription of the electron-dressed M_{ij} of (III.11).

5.3 M_{ij} of (III.11) vs $W_i \otimes W_j$ dressing

Formula (III.11)/(3.11) splits $|\mathcal{M}|^2$ into four blocks M_{11}, M_{12}, M_{21}, M_{22} weighted by the W_i of (3.8). With $B = (q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2$ and $Q = p_6 - p_7$, script `scripts/mumu_mij_check.py` checks each corner separately and the full sum:

$$M_{ij}/B^2 \;\equiv\; S_{\text{naive}} \text{ contracted with } L_{W_i}(q_1, p_1) \otimes L_{W_j}(q_2, p_2), \tag{15}$$

and $\sum_{ij} M_{ij}/B^2$ against $L \otimes L$ ($W_1 = W_2 = 1$). Here $L_{W_1}(q) = 2q^2(g - qq/q^2)$ and $L_{W_2} = 8\, r_\mu r_\nu$ with $r = p - (p{\cdot}q)/q^2\, q$. Electron momenta are massless with $2p_i{\cdot}q_i = q_i^2$. Transcription details, the three-index ε sign convention in M_{12}/M_{21}, and the M_{22} bilinear index order are in Appendix B.

```
python3 scripts/mumu_mij_check.py   # must print PASS: M_ij/B^2 ...
```

This closes the electron-dressed (III.11) layer against the naive poly.

5.4 What is not yet claimed

The checks above are numerical identities (finite-difference Hessian × transcribed M_{ij}). FORM gates the Appendix C identities and the M_{ij} Levi-Civita↔poly transcription: the (C1) toolkit (`mumu_schouten_c1.frm`), the load-bearing reductions (C.4)/(C.8)/(C.9)/(C.13)/(C.14) (`mumu_schouten_c2_c14.f`
`make -C form appC`), and `mumu_mij.frm` (`make -C form mij`; all four Diff$M_{ij} = 0$). Sections 6–7 compare the naive poly to a C evaluator of (3.5); $\sum M_{ij}/B^2$ is gated by `make -C numeric check` / `make -C numeric mij`.

6 Numerical demonstration of the cancellation

Section 5 checked the Ward premise symbolically in FORM. This section shows the *numeric* consequence of evaluating the naive 98-term 4-vector-product polynomial of Section 3 in floating point when that cancellation is nearly exact — the same kind of stress as `kinc2-driver/check_gram_minors.c` for the π^0 Gram minors, but for the μ-pair squared amplitude.

6.1 Stress configuration

Take a fixed mildly virtual two-photon kinematics ($W = 20\,\text{GeV}$, $m_\mu = 0.105658\,\text{GeV}$, $\varepsilon = 10^{-2}$ in $q_z = \sqrt{E^2 + \varepsilon}$) and set

$$e_1 = q_1 + \delta\,\hat{x}, \qquad e_2 = \hat{y}. \tag{16}$$

At $\delta = 0$ one has $e_1 = q_1$, so the squared Ward identity of Sections 3.6 and 5.1 forces $S = 0$ exactly. For small δ, $S = O(\delta^2)$ (or smaller) while individual monomials in the expanded poly remain $O(1)$–$O(W^2)$: the sum is a cancellation between large terms.

The poly is the committed header `numeric/eval_naive_poly.h`, emitted from `scripts/emit_naive_poly_c.py` (same sympy sum as Section 3). The driver `numeric/digit_loss_naive.c` evaluates it in `double` and `long double`, and the same kinematics through the C (3.5) evaluator (`eval_stable_35.h`).

6.2 Scan

Run from `numeric/`:

```
make && make run
```

Representative output ($W = 20\,\text{GeV}$, $\varepsilon = 10^{-2}$):

δ	S (double)	S (long double)	rel. err.	digits
10^{-1}	8.00×10^{-2}	8.00×10^{-2}	3×10^{-12}	11.5
10^{-2}	8.00×10^{-4}	8.00×10^{-4}	3×10^{-10}	9.5
10^{-3}	8.00×10^{-6}	8.00×10^{-6}	6×10^{-9}	8.2
10^{-4}	8.00×10^{-8}	8.00×10^{-8}	6×10^{-7}	6.2
10^{-5}	8.00×10^{-10}	8.00×10^{-10}	1×10^{-4}	3.9
10^{-6}	8.2×10^{-12}	7.9×10^{-12}	3×10^{-2}	1.5
10^{-7}	5.4×10^{-13}	4.7×10^{-14}	~ 10	< 0

Digits of agreement fall steadily as $\delta \to 0$. By $\delta \sim 10^{-7}$ double and long double no longer even share a leading digit: the naive expanded route has lost the answer to cancellation noise. On the same ray the (3.5) value keeps falling toward zero (e.g. $|S_{(3.5)}| \sim 10^{-20}$ at $\delta = 10^{-8}$) while the naive double saturates near 10^{-13}. This is a concrete instance of Vermaseren's warning that substituting numerical 4-vector products into a fully expanded expression is dangerous near the gauge / multiperipheral regime.

6.3 What this section does and does not claim

1. **Claimed here:** evaluating the *naive* 98-term poly in IEEE double near $e_1 = q_1$ loses essentially all significant digits relative to the same poly in long double; the loss tracks the approach to the Ward zero.

2. **Also claimed here:** side-by-side IEEE comparison of the naive poly against the C (3.5) evaluator on this δ-ray (`make -C numeric run`): as $\delta \to 0$ the naive double value saturates near 10^{-13} while (3.5) continues toward zero.

3. **Not claimed here:** (i) that relative double/long-double digits of (3.5) itself stay high all the way to $\delta = 0$ (both forms evaluate a quantity $\to 0$, so relative digits dip; the absolute-value story is the clean claim); (ii) a Monte Carlo fraction of phase-space points that are "bad" (Section 7); (iii) that the near-gauge ray $e_1 = q_1 + \delta\hat{x}$ is identical to the physical $t_1, t_2 \to 0$ multiperipheral peak — it is the cleanest probe of the same cancellation, not a full phase-space survey.

7 Monte Carlo scan

Section 6 showed digit loss along one near-gauge ray. This section asks the Monte Carlo question: over randomly sampled orientations, what *fraction* of points is bad, and as a function of what proximity?

7.1 Setup

Driver: `numeric/mc_digit_loss.c`. Same poly header as Section 6. For each bin, draw $N = 5000$ independent samples of the muon direction (θ, ϕ) and transverse polarization angles; evaluate the poly in `double` and `long double`; call a point **bad** when digits of agreement are < 6. Two ensembles:

1. **phys $\times$ eps.** Physical transverse e_1, e_2 ($e_i \cdot q_i = 0$). Bin by photon virtuality ε with $q_1^2 = q_2^2 = -\varepsilon$ ($\varepsilon \to 0$ is the multiperipheral peak of the paper).

2. **gauge × delta.** $e_1 = q_1 + \delta\, e_T$ with random transverse e_T, fixed $\varepsilon = 10^{-2}$. Bin by $\delta \to 0$ (approach to the Ward surface of Sections 3.6–6).

```
make -C numeric mc
```

7.2 Results

ensemble / bin	frac_bad	mean digits
phys, $\varepsilon = 10^{-1}$	0	26.1
phys, $\varepsilon = 10^{-6}$	0	26.5
phys, $\varepsilon = 10^{-12}$	0	26.2
gauge, $\delta = 10^{-1}$	0	11.8
gauge, $\delta = 10^{-2}$	0	9.7
gauge, $\delta = 10^{-3}$	0.011	7.8
gauge, $\delta = 10^{-4}$	0.69	5.7
gauge, $\delta = 10^{-5}$	0.99	3.8
gauge, $\delta \le 10^{-7}$	1.00	$\lesssim 0.2$

Physical transverse samples stay ≥ 11 digits of agreement at every ε down to 10^{-12}: approaching the multiperipheral peak *alone* does not make this $\gamma^*\gamma^* \to \mu^+\mu^-$ subsystem poly lose precision in IEEE double. The bad fraction rises only when the polarization approaches the Ward surface ($\delta \lesssim 10^{-4}$), matching Section 6 but now as a phase-space rate.

7.3 What this section does and does not claim

1. **Claimed here:** for the naive 98-term poly, catastrophic double/long-double disagreement is a near-gauge phenomenon; its MC fraction tracks δ, not ε.

2. **Also claimed here:** the same ensembles now report `frac_bad` for the C (3.5) evaluator alongside the naive poly (`make -C numeric mc`).

3. **Not claimed here:** (i) that the full $e^+e^- \to e^+e^-\mu^+\mu^-$ Monte Carlo is free of peak-driven loss (electron-line currents can source effective longitudinal admixture near $t_1, t_2 \to 0$); (ii) importance sampling of the paper's multiperipheral weight — this is a uniform orientation probe of the poly, not a cross-section estimate.

8 Conclusion

Vermaseren's warning stands as written; this report does not amend it. What we establish for the isolated subprocess $\gamma^*\gamma^* \to \mu^+\mu^-$ is that the naive expansion into 4-vector products (98-term poly, Section 3) and the Schouten-stabilized route (3.5) are symbolically identical, that the Ward identity holds in Python and FORM (Sections 3.6, 5), and that IEEE double evaluation of the naive poly can lose essentially all digits relative to long double and to (3.5) (Section 6).

The limitation is where that loss appears for free polarizations: Monte Carlo fractions track the Ward surface $e_1 \to q_1$, not soft $t_1, t_2 \to 0$ alone (Section 7). Soft-$|t|$ digit-loss of dressed L⊗L(naive) versus $\sum M_{ij}$ is seen on `gamgam` samples (`make -C kinc2-driver digit-loss-eemm`). Neither replaces a full $e^+e^- \to e^+e^-\mu^+\mu^-$ generator under CDC-era arithmetic.

The algebraic and tooling layers around that finding are closed: naive≡(3.5); $\sum_{ij} M_{ij} \equiv L \otimes L$; C evaluators in the digit-loss / MC drivers; FORM App-C and M_{ij} transcription; `gamgam/peripp` kinematics gate.

A Full naive derivation: subtleties and term dump

A.1 Why the main text stops at one T_{ij}

Hand-reducing all sixteen T_{ij} through (6) is possible but error-prone: the longest chains (e.g. T_{11} and T_{33}) are six-gamma traces with three non-commuting momenta from the propagators, and the mixed $D_1 D_2$ blocks (T_{13}, T_{31}, ...) couple k_1 and k_2. The trustworthy record of that algebra is the program in `mumu_trace_lib.py`, not a second hand transcription. What the appendix adds is (i) the issues that actually broke checks during development, (ii) the per-T_{ij} monomial counts, and (iii) the raw generated dump.

A.2 Bugs and failed checks (kept visible)

Duplicated token in the amplitude Ward script. An early version of `mumu_gauge_check.py` encoded diagram 1's k-piece as the word `['p6','k1','k1','e2']` instead of `['p6','k1','e2']`. The machine then disagreed with the hand reduction in that script's docstring. The hand reduction was right; the word list was wrong. After the fix, machine and hand agree: diagram 1 $\to -\not{\epsilon}_2$, diagram 2 $\to +\not{\epsilon}_2$.

Missing on-shell conditions in the squared Ward script. With $p_6 \cdot p_6$ and $p_7 \cdot p_7$ left free, `mumu_gauge_check_` does *not* return zero (order 200 leftover monomials after substituting $D_i = k_i^2 - m^2$). Imposing $p_6 \cdot p_6 = p_7 \cdot p_7 = m^2$ restores zero. The same substitution was then applied in the naive sum. Re-running the naive script with the substitution present did not change the 98-term count relative to the earlier draft that omitted it in that script alone — but the squared Ward check is the reason the substitution is mandatory for the pipeline as a whole.

Wrong Ward test on $|M|^2$. Substituting $e_1 \to q_1$ into *both* N and $\bar{N}$ makes the squared trace vanish for a trivial reason. That test was rejected; the script substitutes only on N.

A.3 Per-piece structure (from the generator)

Monomial counts per T_{ij} after on-shell substitution, as printed into `scripts/generated/mumu_naive_Tij.txt`:

T_{ij}	denominator	monomials
T_{11}	D_1^2	9
T_{12}	D_1^2	4
T_{13}	$D_1 D_2$	34
T_{14}	$D_1 D_2$	10
T_{21}	D_1^2	4
T_{22}	D_1^2	5
T_{23}	$D_1 D_2$	10
T_{24}	$D_1 D_2$	8
T_{31}	$D_1 D_2$	34
T_{32}	$D_1 D_2$	10
T_{33}	D_2^2	9
T_{34}	D_2^2	4
T_{41}	$D_1 D_2$	10
T_{42}	$D_1 D_2$	8
T_{43}	D_2^2	4
T_{44}	D_2^2	5
sum of row counts		168
collected S		98

The row-count sum (168) exceeds 98 because like terms from different T_{ij} combine when the sixteen pieces are added. The $18+18+62$ split of the collected S by denominator family is checked in the same script and is independent of the row table above.

Observable symmetries in the dump (not assumed in the code, read off after the fact): $T_{12} = T_{21}$, $T_{14} = T_{41}$, $T_{23} = T_{32}$, $T_{24} = T_{42}$, and $T_{13} = T_{31}$. The diagonal blocks T_{11} and T_{33} are $k_1 \leftrightarrow k_2$, $D_1 \leftrightarrow D_2$ images of each other up to the $e_1 \leftrightarrow e_2$ routing of the crossed diagram; the dump should be read with that in mind rather than as sixteen unrelated polynomials.

A.4 Code layout

file	role
`scripts/mumu_trace_lib.py`	shared contraction, N_i definitions, on-shell map
`scripts/mumu_naive_trace.py`	builds all T_{ij} and S; writes the dump file
`scripts/mumu_gauge_check.py`	Ward identity on N (amplitude level)
`scripts/mumu_gauge_check_squared.py`	Ward identity on the squared-trace sum
`scripts/mumu_stable_35_check.py`	naive $\equiv$ paper (3.5); see Appendix B
`scripts/mumu_mij_check.py`	$\sum M_{ij}/B^2 \equiv L \otimes L$; see Appendix B
`scripts/emit_naive_poly_c.py`	emits `numeric/eval_naive_poly.h` for Secs. 6–7
`form/mumu_ward.frm`	FORM squared Ward (`Sg = 0`)
`form/mumu_naive.frm`	FORM Trace4 baseline for the naive $N/\bar{N}$ split
`form/mumu_schouten_c1.frm`	App-C toolkit ($\varepsilon\varepsilon$, biline, Schouten rewrite)
`form/mumu_schouten_c2_c14.frm`	App-C (C.4)/(C.8)/(C.9)/(C.13)/(C.14) for M_{12}/M_{21}
`form/mumu_mij.frm`	(3.11) M_{ij} Levi-Civita form vs closed poly
`numeric/eval_stable_35.h`	C (3.5) evaluator (double / long double)
`numeric/eval_mij.h`	C $\sum M_{ij}/B^2$ evaluator
`numeric/digit_loss_naive.c`	Sec. 6 near-gauge δ ray (naive vs (3.5))
`numeric/mc_digit_loss.c`	Sec. 7 phys$\times\varepsilon$ / gauge$\times\delta$ fractions
`numeric/check_stable_c.c`	fixed-kinematics C (3.5)/M_{ij} vs Python refs
`scripts/README-mumu.md`	run order and explicit non-assumptions
`scripts/generated/mumu_naive_Tij.txt`	machine output included below

Run from `scripts/`:

```
python3 mumu_gauge_check.py
python3 mumu_naive_trace.py
python3 mumu_gauge_check_squared.py
python3 mumu_stable_35_check.py
python3 mumu_mij_check.py
```

Plus `make -C form check`, `make -C numeric run`, and `make -C numeric mc`.

B Verification of (3.5) and M_{ij}

Main-text Sections 5.2 and 5.3 state the claims; this appendix is the detailed record of what was transcribed, what was corrected, and what the scripts actually test.

B.1 Two different Q's in the paper

Vermaseren uses the letter Q in two incompatible ways:

- In (3.5)/(3.6) (subreaction $\gamma^*\gamma^* \to \mu^+\mu^-$), $Q \equiv p_6 - p_7$. Numerically, $D_1 D_2 = (q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2$ holds with that Q, and $D_1 = -q_1{\cdot}q_2 - Q{\cdot}q_1$, $D_2 = -q_1{\cdot}q_2 + Q{\cdot}q_1$ reproduce $k_1^2 - m^2$ and $k_2^2 - m^2$.
- In (3.10)/(III.10) the same letter appears in the denominator factor $(q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2$ with the structural reading $Q = q_1 + q_2$ used in Topic 6's denominator note. That identity is a different algebraic object (asymmetric in t_1).

All checks in this appendix use $Q = p_6 - p_7$, matching (3.5) and the M_{ij} blocks as transcribed from Nucl. Phys. B229 (1983) 355–356.

B.2 Transcribed (3.5) used by the checker

With $\mathrm{Tr}[i{\cdot}j] = \varepsilon^{q_i e_i \mu\nu}\varepsilon_{q_j e_j \nu\mu}$, $Q{\cdot}1{\cdot}2{\cdot}Q = \varepsilon^{q_1 e_1 Q\mu}\varepsilon_{q_2 e_2 \mu Q}$, and $B = (q_1{\cdot}q_2)^2 - (Q{\cdot}q_1)^2$, the implementation is

$$\begin{aligned}|\mathcal{M}|^2_{(3.5)} = B^{-2}\Big\{ & 8B\big(\tfrac{1}{4}(\mathrm{Tr}[1{\cdot}2])^2 - \varepsilon^{q_1 e_1 q_2 e_2}\varepsilon_{q_1 e_1 q_2 e_2}\big) \\ & - 2(Q{\cdot}q_1)^2\,\mathrm{Tr}[1{\cdot}1]\,\mathrm{Tr}[2{\cdot}2] \\ & - 8\big(Q{\cdot}1{\cdot}2{\cdot}Q - \tfrac{1}{2}\mathrm{Tr}[1{\cdot}2](Q^2 + q_1{\cdot}q_2)\big)^2 \\ & - \mathrm{Tr}[1{\cdot}1]\big(\varepsilon^{q_2 e_2 \alpha\beta}\varepsilon_{q_1 Q \alpha\beta}\big)^2 \\ & - \mathrm{Tr}[2{\cdot}2]\big(\varepsilon^{q_1 e_1 \alpha\beta}\varepsilon_{q_2 Q \alpha\beta}\big)^2\Big\}.\end{aligned} \tag{17}$$

Minkowski $(+---)$ reductions used in code (cross-checked against explicit index sums):

$$\mathrm{Tr}[i{\cdot}j] = 2\big((q_i{\cdot}q_j)(e_i{\cdot}e_j) - (q_i{\cdot}e_j)(e_i{\cdot}q_j)\big), \tag{18}$$

$$\varepsilon^{abcd}\varepsilon_{abcd} = \det\big(G(a,b,c,d)\big), \quad G_{ij} = v_i{\cdot}v_j, \tag{19}$$

$$\varepsilon^{ab\mu\nu}\varepsilon_{cd\mu\nu} = -2\big((a{\cdot}c)(b{\cdot}d) - (a{\cdot}d)(b{\cdot}c)\big). \tag{20}$$

The chain $Q{\cdot}1{\cdot}2{\cdot}Q$ is the 3×3 Gram minor of (q_1, e_1, Q) against (q_2, e_2, Q).

B.3 Scanned-sign correction on the last two bilinears

A literal reading of the scanned (3.5) suggests overall *plus* signs in front of the two $\mathrm{Tr}[i{\cdot}i](\varepsilon{\cdot}\varepsilon)^2$ terms. Least-squares fitting of those six schematic coefficients against the naive poly on random samples returned

$$(2,\ -8,\ -2,\ -8,\ -1,\ -1)$$

for $(\mathrm{Tr}^2_{12},\ \varepsilon^4,\ \mathrm{Tr}_{11}\mathrm{Tr}_{22},\ (\cdots)^2,\ \mathrm{Tr}_{11}\varepsilon^2_2,\ \mathrm{Tr}_{22}\varepsilon^2_1)$, i.e. the printed $\frac{1}{4}$ and the -2, -8 coefficients match, but the last two bilinears must be *minus*. Equation (17) uses that correction. Without it, relative errors sit at the 10^{-3}–10^{-2} level; with it, `mumu_stable_35_check.py` reports `PASS` (100 samples: 50 physical-transverse, 50 near-gauge; relative error $< 10^{-8}$, or absolute $< 10^{-11}$ near $S = 0$).

B.4 $M_{ij}/B^2 \equiv W_i \otimes W_j$ method

The point-particle tensor (3.8) splits into $L_{W_1} = 2q^2(g - qq/q^2)$ and $L_{W_2} = 8\,r_\mu r_\nu$, $r = p - (p{\cdot}q)/q^2\,q$. If $S(e_1, e_2) = H_{\mu\nu\rho\sigma}e_1^\mu e_1^\nu e_2^\rho e_2^\sigma$ (schematically), the script recovers the quadratic form in each polarization by finite differences on the coordinate basis,

$$H_{\mu\nu} = \begin{cases} S(e_{(\mu)}) & \mu = \nu, \\ \tfrac{1}{2}\big(S(e_{(\mu)} + e_{(\nu)}) - S(e_{(\mu)}) - S(e_{(\nu)})\big) & \mu \neq \nu, \end{cases}$$

then contracts $H_{\mu\nu}L^{W_i\,\mu\nu}$. Nested in e_1 then e_2, that yields $S_{W_iW_j}$. The falsifiable identities are $M_{ij}/B^2 = S_{W_iW_j}$ for each corner and $\sum_{ij} M_{ij}/B^2 = S_{LL}$ (not $M_{ij}/(q_1^2q_2^2B)^2$: the $(q_1^2q_2^2)^2$ of (3.10) is already carried by the $2q^2$ factors inside each L_{W_1}). Samples use massless p_i with $2p_i{\cdot}q_i = q_i^2$.

Transcription notes (NPB 355–356)

- M_{11}: Levi-Civita-free; transcribed as (21) (prefactor $64\,q_1^2q_2^2$).

- M_{12}/M_{21}: NPB writes $B(-\varepsilon^{p_2q_2q_1\mu}\varepsilon_{p_2q_2q_1\mu}-\cdots)$. The Gram helper `eps3sq` $= -\det G(a,b,c)$ used for three-index contractions has the opposite overall sign relative to that typesetting, so the implementation uses $B\big(\texttt{eps3sq}(p_2,q_2,q1)+\texttt{eps3sq}(p_2,q_2,Q)\big)$ (and the M_{21} swap). The mix bilinears are $\varepsilon^{p_2q_2\mu\nu}\varepsilon_{q_1Q\mu\nu}$ as printed.

- M_{22}: overall 512 times the brace on p. 356. The $\frac{1}{8}\mathrm{Tr}_{11}(\varepsilon^{p_2q_2}\varepsilon_{q_1Q})^2$ terms contract the *second* pair against (q_1,Q) or (q_2,Q), not (p_1,Q) — a naive $(q_i,e_i)\to(p_i,q_i)$ map of (3.5) that also remaps those slots fails at the 10^{-3} level.

PASS gate: relative error $<10^{-5}$ (M_{11}) / $<10^{-4}$ (other corners and full sum) on 30 random multiperipheral samples. Observed worst relative errors: $M_{11}\sim 2\times10^{-7}$, $M_{12}/M_{21}\sim 5\times10^{-7}$, $M_{22}\sim 2\times10^{-5}$, full $\sim 2\times10^{-5}$ (finite-difference Hessian).

$$\begin{aligned}M_{11} = 64\,q_1^2q_2^2\Big\{&B\big[(Q{\cdot}q_1)^2-q_1^2q_2^2+(q_1{\cdot}q_2)^2-Q^2(q_1^2+q_2^2+2m^2)\big]\\&-2(Q{\cdot}q_1)^2(q_1^2+2m^2)(q_2^2+2m^2)\Big\}.\end{aligned}\tag{21}$$

B.5 Inventory of falsifiable gates

gate	command	pass criterion
amplitude Ward	`python3 mumu_gauge_check.py`	prints `ALL TERMS CANCEL`
squared Ward (Python)	`python3 mumu_gauge_check_squared.py`	prints `VANISHES` / exit 0
squared Ward (FORM)	`make -C form check`	`Sg = 0`, App-C, and M_{ij} Diff PASS
naive $\equiv$(3.5)	`python3 mumu_stable_35_check.py`	prints `PASS: naive = (3.5)...`
$\sum M_{ij}\equiv L\otimes L$	`python3 mumu_mij_check.py`	prints `PASS: M_ij/B2...`
digit-loss ray	`make -C numeric run`	naive digits fall; (3.5) keeps falling in $\lvert S\rvert$
MC fraction	`make -C numeric mc`	phys frac_bad $=0$; gauge naive $\to 1$
C (3.5)/M_{ij} refs	`make -C numeric check`	prints `PASS: check_stable_c`
C M_{ij} digits	`make -C numeric mij`	prints `PASS: digit_loss_mij`

C Generated term dump

The following file is produced by `mumu_naive_trace.py`. It is the authoritative listing of every T_{ij} and of S; if the prose and this file disagree, the file wins after a clean re-run of the script.

Listing 1: Generated naive-trace dump (`scripts/generated/mumu_naive_Tij.txt`).

```
# Generated by scripts/mumu_naive_trace.py - do not edit by hand.
# Each T[Ni,Nj] includes its propagator denominator factor.
# On-shell substitutions applied: p6.p6 = p7.p7 = m^2.
# Momentum conservation among {e1,e2,k1,k2,p6,p7} is NOT imposed.
# D1, D2 are kept as independent symbols (not substituted by k^2-m^2).

T[N1,N1]  monomials=9
-4*e1.e1*e2.e2*k1.k1*m**2/D1**2 - 4*e1.e1*e2.e2*k1.k1*p6.p7/D1**2 + 8*e1.e1*e2.e2*k1.p6*k1.p7/D1**2
     - 16*e1.e1*e2.k1*e2.p7*k1.p6/D1**2 + 8*e1.e1*e2.p6*e2.p7*k1.k1/D1**2 - 16*e1.e2*e1.p6*e2.p7*k1
    .k1/D1**2 - 16*e1.k1*e1.p6*e2.e2*k1.p7/D1**2 + 32*e1.k1*e1.p6*e2.k1*e2.p7/D1**2 + 8*e1.p6*e1.p7
    *e2.e2*k1.k1/D1**2

T[N1,N2]  monomials=4
```

```
4*e1.e1*e2.e2*k1.p6*m**2/D1**2 - 4*e1.e1*e2.e2*k1.p7*m**2/D1**2 + 8*e1.e1*e2.k1*e2.p7*m**2/D1**2 -
    8*e1.k1*e1.p6*e2.e2*m**2/D1**2

T[N1,N3]  monomials=34
4*e1.e1*e2.e2*k1.k2*m**2/(D1*D2) + 4*e1.e1*e2.e2*k1.k2*p6.p7/(D1*D2) - 4*e1.e1*e2.e2*k1.p6*k2.p7/(
    D1*D2) - 4*e1.e1*e2.e2*k1.p7*k2.p6/(D1*D2) - 8*e1.e1*e2.k1*e2.k2*m**2/(D1*D2) - 8*e1.e1*e2.k1*
    e2.k2*p6.p7/(D1*D2) + 8*e1.e1*e2.k1*e2.p7*k2.p6/(D1*D2) + 8*e1.e1*e2.k2*e2.p6*k1.p7/(D1*D2) -
    8*e1.e1*e2.p6*e2.p7*k1.k2/(D1*D2) - 8*e1.e2**2*k1.k2*m**2/(D1*D2) - 8*e1.e2**2*k1.k2*p6.p7/(D1*
    D2) + 8*e1.e2**2*k1.p6*k2.p7/(D1*D2) + 8*e1.e2**2*k1.p7*k2.p6/(D1*D2) + 8*e1.e2*e1.k1*e2.k2*m
    **2/(D1*D2) + 8*e1.e2*e1.k1*e2.k2*p6.p7/(D1*D2) - 8*e1.e2*e1.k1*e2.p6*k2.p7/(D1*D2) - 8*e1.e2*
    e1.k1*e2.p7*k2.p6/(D1*D2) + 8*e1.e2*e1.k2*e2.k1*m**2/(D1*D2) + 8*e1.e2*e1.k2*e2.k1*p6.p7/(D1*D2
    ) - 8*e1.e2*e1.k2*e2.p6*k1.p7/(D1*D2) - 8*e1.e2*e1.k2*e2.p7*k1.p6/(D1*D2) - 8*e1.e2*e1.p6*e2.k1
    *k2.p7/(D1*D2) - 8*e1.e2*e1.p6*e2.k2*k1.p7/(D1*D2) + 8*e1.e2*e1.p6*e2.p7*k1.k2/(D1*D2) - 8*e1.
    e2*e1.p7*e2.k1*k2.p6/(D1*D2) - 8*e1.e2*e1.p7*e2.k2*k1.p6/(D1*D2) + 8*e1.e2*e1.p7*e2.p6*k1.k2/(
    D1*D2) - 8*e1.k1*e1.k2*e2.e2*m**2/(D1*D2) - 8*e1.k1*e1.k2*e2.e2*p6.p7/(D1*D2) + 16*e1.k1*e1.k2*
    e2.p6*e2.p7/(D1*D2) + 8*e1.k1*e1.p6*e2.e2*k2.p7/(D1*D2) + 8*e1.k2*e1.p7*e2.e2*k1.p6/(D1*D2) -
    8*e1.p6*e1.p7*e2.e2*k1.k2/(D1*D2) + 16*e1.p6*e1.p7*e2.k1*e2.k2/(D1*D2)

T[N1,N4]  monomials=10
-4*e1.e1*e2.e2*k1.p6*m**2/(D1*D2) + 4*e1.e1*e2.e2*k1.p7*m**2/(D1*D2) - 8*e1.e1*e2.k1*e2.p7*m**2/(D1
    *D2) + 8*e1.e2**2*k1.p6*m**2/(D1*D2) - 8*e1.e2**2*k1.p7*m**2/(D1*D2) - 8*e1.e2*e1.k1*e2.p6*m
    **2/(D1*D2) + 8*e1.e2*e1.k1*e2.p7*m**2/(D1*D2) - 8*e1.e2*e1.p6*e2.k1*m**2/(D1*D2) + 8*e1.e2*e1.
    p7*e2.k1*m**2/(D1*D2) + 8*e1.k1*e1.p6*e2.e2*m**2/(D1*D2)

T[N2,N1]  monomials=4
4*e1.e1*e2.e2*k1.p6*m**2/D1**2 - 4*e1.e1*e2.e2*k1.p7*m**2/D1**2 + 8*e1.e1*e2.k1*e2.p7*m**2/D1**2 -
    8*e1.k1*e1.p6*e2.e2*m**2/D1**2

T[N2,N2]  monomials=5
-4*e1.e1*e2.e2*m**4/D1**2 + 4*e1.e1*e2.e2*m**2*p6.p7/D1**2 - 8*e1.e1*e2.p6*e2.p7*m**2/D1**2 + 16*e1
    .e2*e1.p6*e2.p7*m**2/D1**2 - 8*e1.p6*e1.p7*e2.e2*m**2/D1**2

T[N2,N3]  monomials=10
-4*e1.e1*e2.e2*k2.p6*m**2/(D1*D2) + 4*e1.e1*e2.e2*k2.p7*m**2/(D1*D2) + 8*e1.e1*e2.k2*e2.p6*m**2/(D1
    *D2) + 8*e1.e2**2*k2.p6*m**2/(D1*D2) - 8*e1.e2**2*k2.p7*m**2/(D1*D2) - 8*e1.e2*e1.k2*e2.p6*m
    **2/(D1*D2) + 8*e1.e2*e1.k2*e2.p7*m**2/(D1*D2) - 8*e1.e2*e1.p6*e2.k2*m**2/(D1*D2) + 8*e1.e2*e1.
    p7*e2.k2*m**2/(D1*D2) - 8*e1.k2*e1.p7*e2.e2*m**2/(D1*D2)

T[N2,N4]  monomials=8
4*e1.e1*e2.e2*m**4/(D1*D2) - 4*e1.e1*e2.e2*m**2*p6.p7/(D1*D2) + 8*e1.e1*e2.p6*e2.p7*m**2/(D1*D2) -
    8*e1.e2**2*m**4/(D1*D2) + 8*e1.e2**2*m**2*p6.p7/(D1*D2) - 8*e1.e2*e1.p6*e2.p7*m**2/(D1*D2) - 8*
    e1.e2*e1.p7*e2.p6*m**2/(D1*D2) + 8*e1.p6*e1.p7*e2.e2*m**2/(D1*D2)

T[N3,N1]  monomials=34
4*e1.e1*e2.e2*k1.k2*m**2/(D1*D2) + 4*e1.e1*e2.e2*k1.k2*p6.p7/(D1*D2) - 4*e1.e1*e2.e2*k1.p6*k2.p7/(
    D1*D2) - 4*e1.e1*e2.e2*k1.p7*k2.p6/(D1*D2) - 8*e1.e1*e2.k1*e2.k2*m**2/(D1*D2) - 8*e1.e1*e2.k1*
    e2.k2*p6.p7/(D1*D2) + 8*e1.e1*e2.k1*e2.p7*k2.p6/(D1*D2) + 8*e1.e1*e2.k2*e2.p6*k1.p7/(D1*D2) -
    8*e1.e1*e2.p6*e2.p7*k1.k2/(D1*D2) - 8*e1.e2**2*k1.k2*m**2/(D1*D2) - 8*e1.e2**2*k1.k2*p6.p7/(D1*
    D2) + 8*e1.e2**2*k1.p6*k2.p7/(D1*D2) + 8*e1.e2**2*k1.p7*k2.p6/(D1*D2) + 8*e1.e2*e1.k1*e2.k2*m
    **2/(D1*D2) + 8*e1.e2*e1.k1*e2.k2*p6.p7/(D1*D2) - 8*e1.e2*e1.k1*e2.p6*k2.p7/(D1*D2) - 8*e1.e2*
    e1.k1*e2.p7*k2.p6/(D1*D2) + 8*e1.e2*e1.k2*e2.k1*m**2/(D1*D2) + 8*e1.e2*e1.k2*e2.k1*p6.p7/(D1*D2
    ) - 8*e1.e2*e1.k2*e2.p6*k1.p7/(D1*D2) - 8*e1.e2*e1.k2*e2.p7*k1.p6/(D1*D2) - 8*e1.e2*e1.p6*e2.k1
    *k2.p7/(D1*D2) - 8*e1.e2*e1.p6*e2.k2*k1.p7/(D1*D2) + 8*e1.e2*e1.p6*e2.p7*k1.k2/(D1*D2) - 8*e1.
    e2*e1.p7*e2.k1*k2.p6/(D1*D2) - 8*e1.e2*e1.p7*e2.k2*k1.p6/(D1*D2) + 8*e1.e2*e1.p7*e2.p6*k1.k2/(
    D1*D2) - 8*e1.k1*e1.k2*e2.e2*m**2/(D1*D2) - 8*e1.k1*e1.k2*e2.e2*p6.p7/(D1*D2) + 16*e1.k1*e1.k2*
    e2.p6*e2.p7/(D1*D2) + 8*e1.k1*e1.p6*e2.e2*k2.p7/(D1*D2) + 8*e1.k2*e1.p7*e2.e2*k1.p6/(D1*D2) -
    8*e1.p6*e1.p7*e2.e2*k1.k2/(D1*D2) + 16*e1.p6*e1.p7*e2.k1*e2.k2/(D1*D2)

T[N3,N2]  monomials=10
```

```
-4*e1.e1*e2.e2*k2.p6*m**2/(D1*D2) + 4*e1.e1*e2.e2*k2.p7*m**2/(D1*D2) + 8*e1.e1*e2.k2*e2.p6*m**2/(D1
    *D2) + 8*e1.e2**2*k2.p6*m**2/(D1*D2) - 8*e1.e2**2*k2.p7*m**2/(D1*D2) - 8*e1.e2*e1.k2*e2.p6*m
    **2/(D1*D2) + 8*e1.e2*e1.k2*e2.p7*m**2/(D1*D2) - 8*e1.e2*e1.p6*e2.k2*m**2/(D1*D2) + 8*e1.e2*e1.
    p7*e2.k2*m**2/(D1*D2) - 8*e1.k2*e1.p7*e2.e2*m**2/(D1*D2)

T[N3,N3]  monomials=9
-4*e1.e1*e2.e2*k2.k2*m**2/D2**2 - 4*e1.e1*e2.e2*k2.k2*p6.p7/D2**2 + 8*e1.e1*e2.e2*k2.p6*k2.p7/D2**2
     - 16*e1.e1*e2.k2*e2.p6*k2.p7/D2**2 + 8*e1.e1*e2.p6*e2.p7*k2.k2/D2**2 - 16*e1.e2*e1.p7*e2.p6*k2
    .k2/D2**2 - 16*e1.k2*e1.p7*e2.e2*k2.p6/D2**2 + 32*e1.k2*e1.p7*e2.k2*e2.p6/D2**2 + 8*e1.p6*e1.p7
    *e2.e2*k2.k2/D2**2

T[N3,N4]  monomials=4
4*e1.e1*e2.e2*k2.p6*m**2/D2**2 - 4*e1.e1*e2.e2*k2.p7*m**2/D2**2 - 8*e1.e1*e2.k2*e2.p6*m**2/D2**2 +
    8*e1.k2*e1.p7*e2.e2*m**2/D2**2

T[N4,N1]  monomials=10
-4*e1.e1*e2.e2*k1.p6*m**2/(D1*D2) + 4*e1.e1*e2.e2*k1.p7*m**2/(D1*D2) - 8*e1.e1*e2.k1*e2.p7*m**2/(D1
    *D2) + 8*e1.e2**2*k1.p6*m**2/(D1*D2) - 8*e1.e2**2*k1.p7*m**2/(D1*D2) - 8*e1.e2*e1.k1*e2.p6*m
    **2/(D1*D2) + 8*e1.e2*e1.k1*e2.p7*m**2/(D1*D2) - 8*e1.e2*e1.p6*e2.k1*m**2/(D1*D2) + 8*e1.e2*e1.
    p7*e2.k1*m**2/(D1*D2) + 8*e1.k1*e1.p6*e2.e2*m**2/(D1*D2)

T[N4,N2]  monomials=8
4*e1.e1*e2.e2*m**4/(D1*D2) - 4*e1.e1*e2.e2*m**2*p6.p7/(D1*D2) + 8*e1.e1*e2.p6*e2.p7*m**2/(D1*D2) -
    8*e1.e2**2*m**4/(D1*D2) + 8*e1.e2**2*m**2*p6.p7/(D1*D2) - 8*e1.e2*e1.p6*e2.p7*m**2/(D1*D2) - 8*
    e1.e2*e1.p7*e2.p6*m**2/(D1*D2) + 8*e1.p6*e1.p7*e2.e2*m**2/(D1*D2)

T[N4,N3]  monomials=4
4*e1.e1*e2.e2*k2.p6*m**2/D2**2 - 4*e1.e1*e2.e2*k2.p7*m**2/D2**2 - 8*e1.e1*e2.k2*e2.p6*m**2/D2**2 +
    8*e1.k2*e1.p7*e2.e2*m**2/D2**2

T[N4,N4]  monomials=5
-4*e1.e1*e2.e2*m**4/D2**2 + 4*e1.e1*e2.e2*m**2*p6.p7/D2**2 - 8*e1.e1*e2.p6*e2.p7*m**2/D2**2 + 16*e1
    .e2*e1.p7*e2.p6*m**2/D2**2 - 8*e1.p6*e1.p7*e2.e2*m**2/D2**2

# S_total  monomials=98
-4*e1.e1*e2.e2*k2.k2*m**2/D2**2 - 4*e1.e1*e2.e2*k2.k2*p6.p7/D2**2 + 8*e1.e1*e2.e2*k2.p6*k2.p7/D2**2
     + 8*e1.e1*e2.e2*k2.p6*m**2/D2**2 - 8*e1.e1*e2.e2*k2.p7*m**2/D2**2 - 4*e1.e1*e2.e2*m**4/D2**2 +
     4*e1.e1*e2.e2*m**2*p6.p7/D2**2 - 16*e1.e1*e2.k2*e2.p6*k2.p7/D2**2 - 16*e1.e1*e2.k2*e2.p6*m**2/
    D2**2 + 8*e1.e1*e2.p6*e2.p7*k2.k2/D2**2 - 8*e1.e1*e2.p6*e2.p7*m**2/D2**2 - 16*e1.e2*e1.p7*e2.p6
    *k2.k2/D2**2 + 16*e1.e2*e1.p7*e2.p6*m**2/D2**2 - 16*e1.k2*e1.p7*e2.e2*k2.p6/D2**2 + 16*e1.k2*e1
    .p7*e2.e2*m**2/D2**2 + 32*e1.k2*e1.p7*e2.k2*e2.p6/D2**2 + 8*e1.p6*e1.p7*e2.e2*k2.k2/D2**2 - 8*
    e1.p6*e1.p7*e2.e2*m**2/D2**2 + 8*e1.e1*e2.e2*k1.k2*m**2/(D1*D2) + 8*e1.e1*e2.e2*k1.k2*p6.p7/(D1
    *D2) - 8*e1.e1*e2.e2*k1.p6*k2.p7/(D1*D2) - 8*e1.e1*e2.e2*k1.p6*m**2/(D1*D2) - 8*e1.e1*e2.e2*k1.
    p7*k2.p6/(D1*D2) + 8*e1.e1*e2.e2*k1.p7*m**2/(D1*D2) - 8*e1.e1*e2.e2*k2.p6*m**2/(D1*D2) + 8*e1.
    e1*e2.e2*k2.p7*m**2/(D1*D2) + 8*e1.e1*e2.e2*m**4/(D1*D2) - 8*e1.e1*e2.e2*m**2*p6.p7/(D1*D2) -
    16*e1.e1*e2.k1*e2.k2*m**2/(D1*D2) - 16*e1.e1*e2.k1*e2.k2*p6.p7/(D1*D2) + 16*e1.e1*e2.k1*e2.p7*
    k2.p6/(D1*D2) - 16*e1.e1*e2.k1*e2.p7*m**2/(D1*D2) + 16*e1.e1*e2.k2*e2.p6*k1.p7/(D1*D2) + 16*e1.
    e1*e2.k2*e2.p6*m**2/(D1*D2) - 16*e1.e1*e2.p6*e2.p7*k1.k2/(D1*D2) + 16*e1.e1*e2.p6*e2.p7*m**2/(
    D1*D2) - 16*e1.e2**2*k1.k2*m**2/(D1*D2) - 16*e1.e2**2*k1.k2*p6.p7/(D1*D2) + 16*e1.e2**2*k1.p6*
    k2.p7/(D1*D2) + 16*e1.e2**2*k1.p6*m**2/(D1*D2) + 16*e1.e2**2*k1.p7*k2.p6/(D1*D2) - 16*e1.e2**2*
    k1.p7*m**2/(D1*D2) + 16*e1.e2**2*k2.p6*m**2/(D1*D2) - 16*e1.e2**2*k2.p7*m**2/(D1*D2) - 16*e1.e2
    **2*m**4/(D1*D2) + 16*e1.e2**2*m**2*p6.p7/(D1*D2) + 16*e1.e2*e1.k1*e2.k2*m**2/(D1*D2) + 16*e1.
    e2*e1.k1*e2.k2*p6.p7/(D1*D2) - 16*e1.e2*e1.k1*e2.p6*k2.p7/(D1*D2) - 16*e1.e2*e1.k1*e2.p6*m**2/(
    D1*D2) - 16*e1.e2*e1.k1*e2.p7*k2.p6/(D1*D2) + 16*e1.e2*e1.k1*e2.p7*m**2/(D1*D2) + 16*e1.e2*e1.
    k2*e2.k1*m**2/(D1*D2) + 16*e1.e2*e1.k2*e2.k1*p6.p7/(D1*D2) - 16*e1.e2*e1.k2*e2.p6*k1.p7/(D1*D2)
     - 16*e1.e2*e1.k2*e2.p6*m**2/(D1*D2) - 16*e1.e2*e1.k2*e2.p7*k1.p6/(D1*D2) + 16*e1.e2*e1.k2*e2.
    p7*m**2/(D1*D2) - 16*e1.e2*e1.p6*e2.k1*k2.p7/(D1*D2) - 16*e1.e2*e1.p6*e2.k1*m**2/(D1*D2) - 16*
    e1.e2*e1.p6*e2.k2*k1.p7/(D1*D2) - 16*e1.e2*e1.p6*e2.k2*m**2/(D1*D2) + 16*e1.e2*e1.p6*e2.p7*k1.
    k2/(D1*D2) - 16*e1.e2*e1.p6*e2.p7*m**2/(D1*D2) - 16*e1.e2*e1.p7*e2.k1*k2.p6/(D1*D2) + 16*e1.e2*
```

```
    e1.p7*e2.k1*m**2/(D1*D2) - 16*e1.e2*e1.p7*e2.k2*k1.p6/(D1*D2) + 16*e1.e2*e1.p7*e2.k2*m**2/(D1*
    D2) + 16*e1.e2*e1.p7*e2.p6*k1.k2/(D1*D2) - 16*e1.e2*e1.p7*e2.p6*m**2/(D1*D2) - 16*e1.k1*e1.k2*
    e2.e2*m**2/(D1*D2) - 16*e1.k1*e1.k2*e2.e2*p6.p7/(D1*D2) + 32*e1.k1*e1.k2*e2.p6*e2.p7/(D1*D2) +
    16*e1.k1*e1.p6*e2.e2*k2.p7/(D1*D2) + 16*e1.k1*e1.p6*e2.e2*m**2/(D1*D2) + 16*e1.k2*e1.p7*e2.e2*
    k1.p6/(D1*D2) - 16*e1.k2*e1.p7*e2.e2*m**2/(D1*D2) - 16*e1.p6*e1.p7*e2.e2*k1.k2/(D1*D2) + 16*e1.
    p6*e1.p7*e2.e2*m**2/(D1*D2) + 32*e1.p6*e1.p7*e2.k1*e2.k2/(D1*D2) - 4*e1.e1*e2.e2*k1.k1*m**2/D1
    **2 - 4*e1.e1*e2.e2*k1.k1*p6.p7/D1**2 + 8*e1.e1*e2.e2*k1.p6*k1.p7/D1**2 + 8*e1.e1*e2.e2*k1.p6*m
    **2/D1**2 - 8*e1.e1*e2.e2*k1.p7*m**2/D1**2 - 4*e1.e1*e2.e2*m**4/D1**2 + 4*e1.e1*e2.e2*m**2*p6.
    p7/D1**2 - 16*e1.e1*e2.k1*e2.p7*k1.p6/D1**2 + 16*e1.e1*e2.k1*e2.p7*m**2/D1**2 + 8*e1.e1*e2.p6*
    e2.p7*k1.k1/D1**2 - 8*e1.e1*e2.p6*e2.p7*m**2/D1**2 - 16*e1.e2*e1.p6*e2.p7*k1.k1/D1**2 + 16*e1.
    e2*e1.p6*e2.p7*m**2/D1**2 - 16*e1.k1*e1.p6*e2.e2*k1.p7/D1**2 - 16*e1.k1*e1.p6*e2.e2*m**2/D1**2
    + 32*e1.k1*e1.p6*e2.k1*e2.p7/D1**2 + 8*e1.p6*e1.p7*e2.e2*k1.k1/D1**2 - 8*e1.p6*e1.p7*e2.e2*m
    **2/D1**2

# monomials by denominator family
# 1/D1^2    = 18
# 1/D2^2    = 18
# 1/(D1 D2) = 62
# other     = 0
```